

VOID Engine

Technical Overview & Beginner Reference

Version: 2.1.0 **Target:** .NET 10 **License:** MIT

A plain-English guide to the systems, architecture, extension points, and design choices inside VOID.

Source snapshot

Repository: Shmellyorc/Void

Branch: master

Commit: ee4aab287a8c5459f123fbacfa59c873a0bb77d7

Snapshot date: September 14, 2026

This document is written for developers who may be new to engine architecture. Technical terms are introduced in plain English before the implementation details.

Table of Contents

1. What VOID Is 4

2. Installation and Your First Project 6

3. Configuring Your Game 7

4. The Game Lifecycle 10

5. Frame Timing and Fixed Updates 11

6. Platform, Windowing, and Displays 12

7. Coordinate Systems, Viewports, and Scaling 13

8. Renderer Architecture 15

9. The Built-in OpenGL Backend 16

10. Sprite, Primitive, and Text Rendering 17

11. Render Targets, Shaders, and Post-Processing 18

12. Texture Atlas System 19

13. Camera System 20

14. Asset Management and Virtual File System 21

15. Asset Packer and Command-Line Tool 22

16. Input System 23

17. Audio System 24

18. Resource Ownership and Dispose 25

19. Threading and Background Work 27

20. LDtk, Spritesheets, Fonts, and Content Helpers 28

21. Save and Load System 29

22. Coroutines, Tweens, and Easing	30
23. State Machines	31
24. Beacon Messaging	32
25. Pathfinding and Flow Fields	33
26. Logging and Crash Handling	34
27. Math and Utility Types	35
28. Discovery, Plugins, and Extensibility	36
29. Performance Model and Hot Paths	37
30. What VOID Deliberately Does Not Try to Be	38
31. Packages, Dependencies, and Platform Support	39
32. Building and Shipping a Game	40
33. VOID 2.1.0 Release Highlights	42
34. Migration Notes: Older VOID to 2.1.0	43
35. Beginner Workflow: How the Systems Fit Together	44
36. Common VOID Recipes	45
37. Troubleshooting and FAQ	47
38. Source Map: Where to Look in the Repository	49
39. Glossary	50

CHAPTER 1

1. What VOID Is

VOID is a modular 2D game framework for C# and .NET. It supplies common game systems while deliberately leaving game-specific rules and large all-in-one subsystems to the developer.

Plain English

Think of VOID as a toolbox and foundation, not a visual game editor. It gives you windowing, rendering, input, assets, audio, saving, pathfinding, coroutines, logging, and related infrastructure so you can spend more time on the actual game.

Core idea

VOID follows a simple design rule: provide useful defaults without pretending those defaults fit every game. The built-in systems are meant to be usable immediately, but important areas expose public extension contracts so a developer can replace or extend them without forking the engine.

- **2D first:** the engine focuses on a practical 2D workflow.
- **Framework, not giant editor:** code remains central to the way a project is built.
- **Extensible:** renderer backends, asset sources, atlas packers, log sinks, cameras, batchers, render targets, and save types expose extension points.
- **Cross-platform desktop:** the current supported platforms are Windows, macOS, and Linux.
- **Open source:** VOID is MIT licensed with no royalties or engine fees.

At a glance

Area	Current VOID 2.1.0 choice
Runtime	.NET 10
Platform layer	SDL3-CS
Built-in renderer	Silk.NET.OpenGL
Audio backend	Silk.NET.OpenAL / OpenAL Soft
Image decoding	StbImageSharp
Sound decoding	NAudio.SoundFile
Asset archives	Void.Packer
License	MIT

Your first 30 minutes with VOID

You do not need to read this manual front to back before writing a game. A useful first pass is: What VOID Is, Installation and Your First Project, Configuring Your Game, The Game Lifecycle, Coordinate Systems, Asset Management, Input, Audio, and Sprite Rendering. Then build something small and return to the specialized chapters when you actually need them.

Recommended learning path

Start with configuration, loading one texture, reading one input action, and drawing one sprite. Renderer backends, atlas defragmentation, discovery, custom mounts, and other extension points can wait until your project gives you a reason to care about them.

What changed from the older spec

The older document described an SFML.NET-based engine. That is no longer the architecture. VOID 2.x removed SFML, introduced renderer-neutral graphics contracts, moved the platform layer to SDL3, uses a built-in OpenGL renderer, and moved audio to OpenAL. This document treats the current code as the source of truth.

CHAPTER 2

2. Installation and Your First Project

VOID can be installed as a NuGet package or through its project template. The template is the easiest starting point for someone learning the framework.

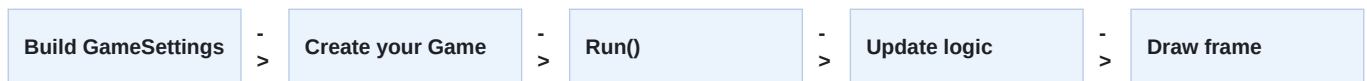
Fastest route

```
dotnet new install Void.Templates
dotnet new voidgame -n MyGame
cd MyGame
dotnet run
```

The template creates a small project with the engine reference and a basic game class. If you prefer to start from a normal console project, add the engine package directly.

```
dotnet new console -n MyGame
cd MyGame
dotnet add package Void.Engine
```

The minimum mental model



Beginner takeaway

Most projects begin with two things: a GameSettings configuration and a class derived from Game. From there, VOID calls your lifecycle methods at the appropriate time.

Minimal game class

```
public sealed class MyGame : Game
{
    public MyGame(GameSettings settings) : base(settings) { }

    protected override void OnEnter() { }
    protected override void OnUpdate(FrameTime frameTime) { }
    protected override void OnDraw(FrameTime frameTime) { }
    protected override void OnExit() { }
}
```

CHAPTER 3

3. Configuring Your Game

GameSettings is the startup configuration for a VOID application. It is a fluent singleton: configure it, call `Build()`, then pass the finalized settings into your Game.

Plain English

GameSettings answers the questions VOID must know before the game starts: what the window should look like, what internal resolution to render at, how timing behaves, which renderer to create, how large the audio pool is, where application files live, and how several engine systems should be sized.

A practical starter configuration

```
var settings = GameSettings.Instance
    .SetAppCompany("MyStudio")
    .SetAppName("MyGame")
    .SetAppTitle("My Game")
    .SetWindow(1280, 720)
    .SetViewport(320, 180)
    .SetVsync(true)
    .Build();

using var game = new MyGame(settings);
game.Run();
```

Window size is not viewport size

`SetWindow` controls the operating-system window size. `SetViewport` controls the game's internal render resolution. They can be different. A 320 x 180 viewport can be presented inside a 1280 x 720 window while game logic and pixel-art rendering continue to think in the smaller coordinate space.

Important defaults after Build()

Setting	Default when not set
Window	1280 x 720
Viewport	320 x 180
Window scaling	Fit
VSync	Enabled
Fixed timestep	Enabled
Target update interval	1/60 second
Maximum delta	0.1 second
Input dead zone	0.15
Ignore input while unfocused	Enabled
Sprite batch capacity	1024
Primitive batch capacity	4096
Default sort mode	BackToFront
Default blend mode	Alpha
Atlas pages	4 pages at 2048 x 2048
Atlas packer	SkylinePacker
Atlas defrag threshold	0.3
Atlas defrag moves per frame	10
Audio instance limit	128
OpenGL version	3.3
Supersampling	4
Minimum log level	Info
Max log file size	10 MB
Retained log files	10
Asset eviction	30 minutes unused
Asset check interval	1 minute
Application version	1.0.0.0
Log / Save / Config / Temp folders	Logs / Saves / Config / Temp

Settings you normally change first

- **Identity:** AppCompany, AppName, AppTitle, and AppVersion.
- **Presentation:** window size, viewport size, scale mode, display, fullscreen style, and VSync.
- **Timing:** fixed versus variable timestep, target elapsed time, and maximum delta.
- **Content:** content root and application-data behavior.
- **Capacity:** audio limit, atlas page size/count, and batch capacities when the defaults no longer fit your game.
- **Renderer:** use the built-in OpenGL default or provide a custom renderer factory.

Build finalizes the configuration

Build() validates required values, finds a conventional Content or Assets directory when no content root was provided, fills in defaults, and marks the settings initialized. Calling Build() again returns the already-finalized instance instead of rebuilding the configuration.

Beginner rule

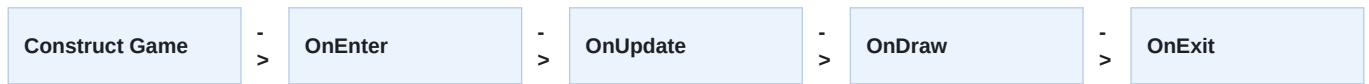
Change startup settings before constructing Game. Runtime window changes belong to Window and ApplyChanges(), not to rebuilding GameSettings after the engine is already running.

CHAPTER 4

4. The Game Lifecycle

The Game class owns the main application lifecycle, window, frame timing, and main loop. Your game plugs into that loop through protected callbacks.

Lifecycle in plain English



OnEnter is the place for startup work that belongs after the engine has initialized. **OnUpdate** is for game logic. **OnDraw** submits rendering. **OnExit** handles game-specific shutdown.

What Game owns for you

- The main Window instance and selected renderer backend.
- A FrameTime object used by the game loop and engine systems.
- Application folders for logs, saves, configuration, and temporary data.
- A built-in SpriteFont for simple text and debugging.
- Default console and file logging sinks.
- Graceful shutdown checks so updates and drawing do not continue after the window closes.

Why this matters

A game loop is the heartbeat of a game. VOID owns the repetitive plumbing while still letting your game decide what update and draw actually mean.

CHAPTER 5

5. Frame Timing and Fixed Updates

FrameTime is VOID's timing model. It tracks elapsed time, fixed-step accumulation, interpolation, time scaling, FPS, and slow-frame information.

Two common timing modes

Mode	What happens
Fixed timestep	Game logic updates use a configured fixed interval. A slow rendered frame can cause multiple fixed updates so simulation time catches up.
Variable timestep	Update and rendering both use the actual elapsed frame time.

VOID 2.1.0 timing behavior

- **During fixed updates:** DeltaTime uses the configured fixed interval, adjusted by TimeScale.
- **During rendering:** DeltaTime uses the actual rendered-frame elapsed time, adjusted by TimeScale.
- **UnscaledDeltaTime:** exposes time before TimeScale is applied.
- **Alpha:** gives an interpolation factor from 0 to 1 in fixed-step mode.
- **FPS and slow-frame tracking:** remain available from the same FrameTime object.

Common beginner mistake

Do not assume one update always equals one rendered frame. In fixed timestep mode, VOID may run zero, one, or several fixed updates before drawing the next frame.

CHAPTER 6

6. Platform, Windowing, and Displays

SDL3 is VOID's platform layer. It owns desktop window creation, displays, fullscreen behavior, events, and the platform-side pieces required by rendering and input.

Window modes and scaling

Feature	Options
Window mode	Windowed, Borderless, Fullscreen
Viewport presentation	Stretch, PixelPerfect, Fit, Fill, None
Fullscreen	Desktop-style and exclusive fullscreen behavior through current settings
Displays	Display IDs, modes, work areas, refresh rates, content scale, and runtime display selection
VSync	Configured at startup and changeable through queued window settings

Window changes use a deferred pattern: setter calls `queue` changes, then `ApplyChanges()` applies them together. That keeps several related window changes from being partially applied one at a time.

Linux backend choice

VOID reports native window capabilities and supports Linux backend preferences including Auto, X11, Wayland, and an X11-first fallback mode. This matters because some window operations and native handles differ between X11 and Wayland.

Native handles

Custom renderers sometimes need the operating system's native window handle. `IRendererContext` exposes safe TryGet-style access for supported handles, including Windows, X11/XWayland, native Wayland, and Cocoa cases. The renderer borrows those handles; it does not own the window.

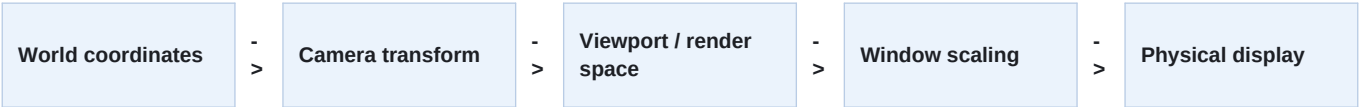
CHAPTER 7

7. Coordinate Systems, Viewports, and Scaling

VOID uses several coordinate spaces that sound similar but solve different problems. Understanding the path from world coordinates to the physical window prevents many beginner rendering bugs.

The short version

Game objects usually live in world or viewport coordinates. A camera transforms the world view. The viewport is the internal render resolution. The window is the physical desktop surface that finally presents that viewport.



The main spaces

Space	What it means
World	Positions used by game objects, maps, enemies, particles, and other scene content.
Camera / view	The transformed view of the world after camera position, rotation, zoom, or custom camera behavior.
Viewport	The internal resolution VOID renders for the game, configured by SetViewport.
Window	The SDL desktop window, configured by SetWindow and runtime Window changes.
Display	The monitor and its physical desktop mode, refresh rate, content scale, and work area.

Example: 320 x 180 inside 1280 x 720

If the viewport is 320 x 180 and the window is 1280 x 720, the window is four times larger in each dimension. With an appropriate scale mode, one viewport pixel can become a 4 x 4 block of window pixels. Your player can still be positioned at x=160, y=90 in the game while appearing at the center of the much larger desktop window.

Scale modes

Mode	Idea
Fit	Keep the viewport aspect ratio and fit the entire image inside the window. Unused space can remain around it.
Fill	Keep the aspect ratio but fill the window, which can crop part of the viewport.
Stretch	Stretch independently to the window dimensions, which can distort the aspect ratio.
PixelPerfect	Prefer integer scaling so viewport pixels remain crisp when possible.
None	Do not apply the normal automatic viewport scaling behavior.

ScreenToWorld and WorldToScreen

BaseCamera provides conversions in both directions. WorldToScreen is useful when a world object needs a screen-space marker or UI label. ScreenToWorld is useful when mouse or pointer coordinates need to become a position inside the game world.

Common bug

If a mouse position, sprite position, and camera position all use different spaces, the math can look correct while the result is wrong. Decide which coordinate space each value belongs to before adding or subtracting them.

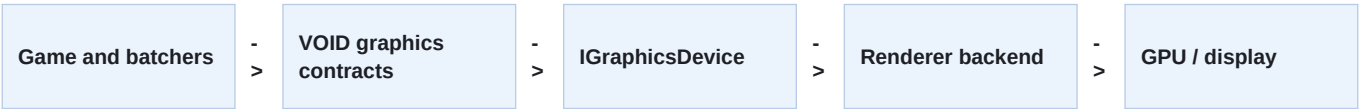
CHAPTER 8

8. Renderer Architecture

The renderer is one of VOID 2.x's biggest architectural changes. Game code no longer depends on OpenGL or any other graphics API directly.

Plain English

A renderer backend is the part that talks to the graphics API. VOID ships an OpenGL backend, but the public engine contracts are designed so another developer can supply a different backend without rewriting the higher-level game framework.



The three important contracts

Contract	Job
IRendererBackend	Owns the graphics API implementation, frame lifecycle, presentation, capabilities, device, and default 2D shader.
IGraphicsDevice	Creates and updates renderer-neutral GPU resources such as buffers, textures, shaders, and render targets, then submits draw commands.
IRendererContext	Provides window/platform services, function loading, and borrowed native handles needed by a backend.

Selecting a custom renderer

```
var settings = GameSettings.Instance
    .SetRenderer(() => new MyRenderer())
    .Build();
```

A custom renderer can target APIs such as Vulkan, Direct3D, Metal, another OpenGL implementation, or a specialized renderer. VOID itself only ships the built-in OpenGL backend.

CHAPTER 9

9. The Built-in OpenGL Backend

VOID's default renderer uses Silk.NET.OpenGL. It is an implementation of the renderer contracts, not the definition of the engine's graphics API.

What the backend owns

- OpenGL context and function access provided through the SDL-created window context.
- GPU buffers, textures, shader programs, and render targets.
- The default 2D shader used by the built-in batching path.
- Capability reporting such as texture limits.
- Presentation and resize behavior.
- A shared OpenGL state cache that avoids redundant binds and state changes in hot paths.

Renderer-neutral resources

Higher-level systems deal with interfaces such as `IGraphicsBuffer`, `IGraphicsTexture`, `IGraphicsShaderProgram`, and `IGraphicsRenderTarget`. That separation is what keeps `SpriteBatcher`, `PrimitiveBatcher`, atlasing, post-processing, and game-facing code from becoming hard-wired to OpenGL.

Important distinction

Renderer-neutral does not mean every graphics API behaves identically. A third-party backend still has to translate VOID's resource and draw contracts into the rules of its own API.

CHAPTER 10

10. Sprite, Primitive, and Text Rendering

VOID provides two main high-level batchers: SpriteBatcher for textured 2D work and PrimitiveBatcher for geometry.

SpriteBatcher

- Batches sprites as indexed quads using 4 unique vertices and 6 indices per sprite.
- Can group compatible commands by texture after the requested depth sort.
- Can use the texture atlas automatically or bypass it when needed.
- Supports bitmap-font text, nine-patch UI geometry, rotation, scaling, origins, effects, tinting, depth, and cameras.
- Supports 9 text alignment positions and word or character wrapping.
- Uses camera bounds for coarse draw culling when a camera is attached.

PrimitiveBatcher

PrimitiveBatcher draws points, lines, rectangles, circles, polygons, and triangles. It writes primitive vertices into a CPU-side stream and submits compatible topology together when it is safe to do so. Strip and fan topology stays separate so unrelated shapes cannot accidentally connect.

A small drawing example

```
using var sprites = new SpriteBatcher();
sprites.Begin(camera: camera);
sprites.Draw(texture, new Vect2(32, 32), Color.White);
sprites.DrawText(font, "Hello VOID", new Vect2(8, 8), Color.White);
sprites.End();
```

CHAPTER 11

11. Render Targets, Shaders, and Post-Processing

These systems let you render into a texture instead of directly to the window, then use that result in later rendering passes.

Render target

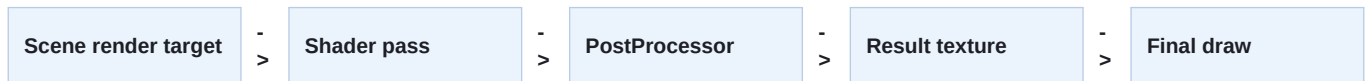
A render target is an off-screen surface. Games commonly use one for a world layer, lighting pass, minimap, low-resolution pixel-art viewport, or any effect that needs the result of an earlier draw pass.

Custom shaders

VOID exposes renderer-neutral shader lifecycle and texture-binding semantics. The built-in 2D path supplies the view-projection matrix, whether a texture is active, texture size, and the primary texture. Backend implementations are responsible for preserving those semantics in their own API.

PostProcessor

PostProcessor is a convenience wrapper that takes the texture produced by a source render target, draws it through a shader, and stores the processed result in a pooled render target. This is the basic pattern behind effects such as tinting, distortion, blur-like passes, color grading, or screen transitions.



CHAPTER 12

12. Texture Atlas System

A texture atlas combines many smaller source regions into larger GPU textures so sprites can be drawn with fewer texture switches.

Plain English

Changing textures can interrupt batching. An atlas reduces those changes by copying frequently used sprite regions into shared pages.

Current default configuration

Setting	Default
Page size	2048 x 2048 pixels
Page count	4
Default packer	SkylinePacker

How VOID 2.x handles atlas pages

- Pages are renderer-neutral and backed by IGraphicsTexture resources from the active renderer.
- Each page keeps a CPU-side RGBA8 shadow copy so GPU resources can be recreated if the graphics device changes.
- Pages and graphics resources are created lazily when needed.
- Texture regions are cached by texture identifier and source rectangle.
- LRU tracking helps recover space by evicting least-recently-used entries.
- Incremental defragmentation can move regions over time instead of forcing a large one-frame rebuild.
- Custom packers implement IAtlasPacker and their results are validated for bounds, overlap, and layout correctness.

Metrics

AtlasMetrics reports used pages, total pages, used pixel area, total pixel area, percentage full, live texture-region count, and eviction count. The older TotalSpaceBytes and UsedSpaceBytes names remain for compatibility but represent pixel area, not bytes.

CHAPTER 13

13. Camera System

VOID 2.1.0 has a matrix-driven camera system that is extensible without coupling the rendering path to `System.Numerics` or a particular renderer.

BaseCamera

- Provides a view-projection matrix and caches its inverse.
- Supports `ScreenToWorld` and `WorldToScreen` coordinate conversion.
- Calculates conservative world-space view bounds for culling, including rotated cameras.
- Exposes protected `OnUpdate(FrameTime)` for custom follow, shake, smoothing, and transition behavior.
- Updates automatically when attached to a batcher, at most once per rendered frame even when several batchers share the camera.

Why automatic camera updates matter

Before 2.1.0, game code could be responsible for manually updating camera behavior. Now the batcher-driven rendering path updates an attached camera using the rendered-frame timing context. This reduces duplicate updates and keeps camera motion aligned with rendering.

Beginner takeaway

A camera changes how world coordinates are transformed into the screen. It does not move every game object. The objects stay in world space; the camera changes the view of that world.

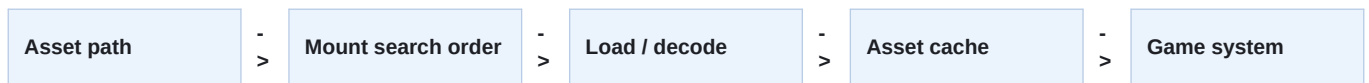
CHAPTER 14

14. Asset Management and Virtual File System

AssetManager is the main entry point for loading and caching game content. It searches an ordered list of mounts instead of assuming every asset lives in one normal folder.

What is a mount?

A mount is a source of files. The normal filesystem is one mount. A packed archive can be another. A game can add a custom IMount for a different source. Mounts are searched from first to last, so high-priority mounts can override lower-priority content.



Built-in asset types

The current AssetManager has built-in loaders for Texture, LDtkMap, SpriteFont, Spritesheet, Sound, and Shader. Developers can register additional asset types with custom extensions and factories.

Caching and eviction

Loaded IAsset instances are cached. Repeated requests can reuse the same asset instead of decoding it every time. The asset system tracks access time so unused content can be evicted according to the engine settings.

```
var assets = AssetManager.Instance;  
Texture player = assets.Load<Texture>("Sprites/player.png");  
  
var pack = assets.LoadPack("Packs/game.pack");  
assets.AddMountToStart(pack);
```

CHAPTER 15

15. Asset Packer and Command-Line Tool

Void.Packer is a separate .NET 10 library for packaging game assets into SolidPack archives. Void.Packer.CLI exposes the same workflow from a terminal.

Why pack assets?

A pack gives a shipped game a smaller number of content files to manage and can add authenticated encryption, compression, integrity checks, and streaming access. It is useful for deployment and for discouraging casual asset extraction.

Security reality

Client-side asset encryption is not perfect DRM. A determined attacker who controls the machine running the game can eventually recover content. VOID's packer is designed to make casual extraction harder while keeping runtime access practical.

Current pack features

- AES-GCM authenticated encryption.
- Separate handling for metadata and file data.
- Per-file integrity verification.
- Adaptive compression.
- Configurable chunked encryption and stream-based disk reads.
- Incremental updates and concurrent read support.
- Multiple-pack output when configured file limits require splitting.
- Non-throwing TryLoadPack helpers.

CLI examples

```
dotnet tool install --global Void.Packer.CLI

void-packer build -c Content/ -o Packs/
void-packer verify --pack Packs/GameAssets.pack
void-packer list --pack Packs/GameAssets.pack --detailed
void-packer extract --pack Packs/GameAssets.pack -o Extracted/
```

CHAPTER 16

16. Input System

VOID's input layer is SDL3-backed and snapshot-oriented. Keyboard, mouse, and gamepad state can be queried directly or combined into named InputAction bindings.

Plain English

A snapshot is a frozen picture of input for one update. Asking for the snapshot again does not advance input or change whether a button counts as newly pressed.

InputAction

- Actions can use string or enum names.
- One action can contain several keyboard, mouse, and gamepad bindings.
- The engine updates action state once per game update.
- `GetState()` returns the already-created snapshot. It does not poll devices and does not perform its own update.
- `TryGetAction` provides a non-throwing lookup path.
- Missing actions and default `InputActionState` values are treated as released / Up rather than as errors.

```
InputAction.AddAction("Jump")
    .AddKey(KeyboardKey.Space)
    .AddGamepad(GamepadButton.A);

var input = InputAction.GetState();
if (input.IsJustPressed("Jump"))
    StartJump();
```

Gamepads

`GamepadState` is an immutable snapshot with digital buttons, triggers, thumbsticks, connection state, and `GetForce()` helpers. VOID includes an SDL controller mapping database and exposes virtual stick directions so an action can treat a stick direction like another binding.

CHAPTER 17

17. Audio System

VOID 2.x uses Silk.NET.OpenAL with OpenAL Soft for playback and NAudio.SoundFile for decoding supported sound formats.

The sound pool

SoundInstancePool pre-allocates a configurable number of SoundInstance objects. Pooling avoids constantly creating and destroying playback objects during gameplay. The configured audio limit must be at least 32.

- Active sounds are updated on a background task at approximately 60 Hz.
- When the pool is exhausted it first reuses completed sounds, can steal a lower-priority active sound, then falls back to the longest-playing active sound.
- Sound instances expose playback controls such as volume, pan, pitch, looping, priority, and events.
- The engine README also exposes category volume support for groups of sounds.

What voice stealing means

A game cannot play an unlimited number of simultaneous sounds. Voice stealing is the policy used to decide which existing sound may be recycled when the pool is full.

CHAPTER 18

18. Resource Ownership and Dispose

C# garbage collection manages normal managed memory, but game engines also own native handles, GPU resources, audio resources, file handles, pools, and other objects whose lifetime should be explicit.

Plain English

Garbage collection does not mean every engine resource can be ignored forever. If an object implements IDisposable because it owns something outside ordinary managed memory, release it when your code owns that object and is finished with it.

The ownership question

Before disposing an object, ask one thing: who created it and who promises to own its lifetime? A resource returned from a cache or manager may be shared. A batcher, post-processor, custom GPU resource, or manually created object may belong directly to your game code. Shared and pooled objects should be returned or released through the system that owns them rather than disposed blindly.

Situation	Normal rule
Game object created with using	Dispose happens when the using scope ends.
SpriteBatcher / PrimitiveBatcher you created	Dispose it when the owner shuts down.
PostProcessor you created	Dispose it; its internal batcher is released and its pooled target is returned.
Low-level graphics resource you created	Dispose it through its renderer-neutral resource object when no longer needed.
AssetManager cached asset	Treat the asset manager as the lifetime owner unless the specific API says ownership was transferred.
Pooled resource	Return it to the pool or owner instead of destroying it directly.

Why GPU and audio resources are different

A Texture object may lead to GPU texture storage. A Sound may own decoded or OpenAL-side resources. A render target can own several graphics objects. Those resources exist outside the normal managed heap, so deterministic cleanup matters even though the C# wrapper object itself is managed.

A simple ownership pattern

```
public sealed class PlayScreen : IDisposable
{
    private readonly SpriteBatcher _sprites = new();
    private readonly PostProcessor _post = new();

    public void Dispose()
    {
        _post.Dispose();
        _sprites.Dispose();
    }
}
```

Do not dispose shared things casually

If a manager gave you a cached asset that other code may also be using, disposing the shared asset yourself can break those users. Follow the ownership contract of the system that produced the object.

CHAPTER 19

19. Threading and Background Work

Most normal VOID gameplay code should be treated as main-thread code unless a subsystem explicitly documents background behavior. A few engine services perform work away from the main game loop.

Main-thread mental model

- Game lifecycle callbacks such as `OnUpdate` and `OnDraw` form the normal place for gameplay state changes.
- Renderer and graphics resource work should be treated as belonging to the rendering context unless a backend explicitly provides another contract.
- `InputAction` snapshots are updated by the game update flow and should be read as state for that update.
- Coroutines are not threads. They resume from the game update flow and do not run in parallel.

Known background systems

System	Background behavior
Audio	<code>SoundInstancePool</code> updates active instances on a background task at about 60 Hz.
Logging	Logger queues work so normal log calls do not synchronously perform all file output.
Your own Tasks	Allowed by .NET, but crossing back into gameplay or renderer-owned state requires the same care as any multithreaded application.

Safe rule for beginners

Do heavy CPU or file work in a background task only when it actually helps, then hand the result back to the main game flow before mutating renderer-owned objects or normal game state. Avoid assuming a collection is thread-safe just because one VOID subsystem internally uses a worker task.

Beacons are synchronous

`BeaconManager.Publish` runs subscribers on the thread that called `Publish`. Publishing from a background task therefore runs those subscribers on that background task too. Only do that when every subscriber is designed for it.

CHAPTER 20

20. LDtk, Spritesheets, Fonts, and Content Helpers

These loaders turn common external content formats into game-friendly runtime assets.

LDtk

LDtkMap parses an LDtk project and builds caches for levels, layers, entities, and tilesets. Common lookups can use Get methods when failure is exceptional or TryGet methods when missing data is normal. Recent cleanup removed exception-driven behavior from optional LDtk setting lookups.

Spritesheets

Spritesheet assets parse named bounds, nine-patch regions, and pivot values from JSON slice metadata. Names are hashed for fast lookup. Parsed entries can remain cached across unload/reload cycles so the JSON does not need to be reparsed every time.

Fonts and text

VOID supports sprite-based fonts and a built-in embedded font for simple text and debugging. SpriteBatcher handles glyph drawing, alignment, wrapping, tabs, carriage-return behavior, camera culling, and atlas participation.

CHAPTER 21

21. Save and Load System

The save system is based on a game-defined `ContentTypeWriterReader`. Your code decides which fields are written and read; VOID handles the file format, validation, optional encryption, and optional compression.

What a save contains

- A VOID file signature and application version hash.
- A type manifest that verifies the order and types expected by the current schema.
- Optional Deflate compression.
- Optional AES-GCM authenticated encryption.
- Game-defined serialized data written through `ContentWriter` and read through `ContentReader`.

Try-style APIs

`TrySave` and `TryLoad` return false with a `SaveError` value for normal failure cases such as invalid paths, missing files, wrong application version, wrong key, corrupt data, or manifest mismatch. Throwing `Save` and `Load` wrappers are available when failure should be exceptional.

Why the manifest exists

If code writes `Name`, `Level`, `Position` but later tries to read those fields in a different order or as different types, the manifest can detect the mismatch instead of silently returning nonsense data.

CHAPTER 22

22. Coroutines, Tweens, and Easing

Coroutines are frame-driven IEnumerator routines. They let game logic pause and resume over several updates without blocking the game thread.

Coroutines are not threads

A coroutine runs a little bit, yields, then continues during a later game update. It is useful for delays, sequences, transitions, scripted behavior, and animation flow. It does not run in parallel like a Task or operating-system thread.

- Yield float, double, or int values to wait that many scaled seconds.
- Yield another IEnumerator to run a nested child and resume the parent after it finishes.
- Yield null or another unrecognized value to resume on a later manager update without a timed delay.
- CoroutineHandle can query or stop a running root coroutine.
- Stopping a root coroutine stops and disposes its active child chain safely.
- Starting and stopping coroutines during a manager update is supported.

Tweens and easing

VOID includes tween routines such as standard, delayed, loop, ping-pong, pulse, speed, and callback tweens. Easing.Ease provides 41 easing curves across linear, polynomial, sine, exponential, circular, back, elastic, and bounce families.

CHAPTER 23

23. State Machines

StateMachine is a coroutine-based finite state machine for systems that should be in one named state at a time, such as an enemy AI, menu flow, animation controller, or game mode.

How it works

States are registered as IEnumerator factories. A state can request a transition by yielding the name of another registered state. The state machine also tracks history, supports pause/resume behavior, and exposes enter, exit, and changed callbacks.

```
var machine = new StateMachine()  
    .AddState("Idle", Idle)  
    .AddState("Active", Active)  
    .OnChanged((from, to) => Console.WriteLine($"{from} -> {to}"));  
  
machine.ChangeState("Idle");
```

Good fit

Use a state machine when the problem is naturally expressed as named modes with transitions. Do not force every gameplay system into a state machine just because one is available.

CHAPTER 24

24. Beacon Messaging

BeaconManager is VOID's topic-based publish/subscribe system for decoupling parts of a game that need to communicate without holding direct references to each other.

Example idea

A Player object can publish "PlayerMoved". A minimap, tutorial system, or achievement tracker can subscribe to that topic. The player does not need to know those systems exist.

- Topics can be strings or enums.
- Several callbacks can subscribe to the same topic.
- Publishing is synchronous on the thread that calls Publish.
- Subscription changes are synchronized while publishing uses a fast lookup of the current multicast delegate.
- BeaconHandle exposes read-only payload access and safe typed Get/TryGet helpers.
- 2.1.0 adds delayed publishing through the coroutine system and typed payload extensions for one through five values.

Synchronous means immediate

When you call Publish, subscribers run before Publish returns. A slow subscriber therefore makes the publisher slow too. This is useful to remember when deciding what work belongs inside a beacon handler.

CHAPTER 25

25. Pathfinding and Flow Fields

AStar2D stores a 2D navigation graph and can answer individual path queries or build a flow field for many agents moving toward the same goal.

Algorithms

Algorithm	Best used for
A*	Normal weighted pathfinding with a heuristic. Usually the default choice.
Dijkstra	Weighted pathfinding when you do not want a heuristic.
BFS	Unweighted graph traversal where edge weights do not matter.
Greedy Best-First	Fast heuristic-directed search that does not guarantee the best path.
Flow field	Many agents sharing one destination. Agents can follow precomputed direction/cost information.

Heuristics and diagonals

Built-in heuristics include None, Manhattan, Euclidean, Octile, and Chebyshev. Diagonal rules can allow all diagonals, reject them, or check adjacent corner points. Custom edge cost, heuristic, and neighbor filtering callbacks can adapt the graph to a game's rules.

The pathfinder reuses backing arrays and its open-set storage across searches to reduce allocations in repeated pathfinding workloads.

CHAPTER 26

26. Logging and Crash Handling

Logger is an asynchronous process-wide logging service with severity levels, categories, formatted messages, and pluggable output sinks.

Log levels

The built-in levels are Debug, Info, Warning, Error, Fatal, and None. Messages below the configured minimum level are discarded before string formatting, avoiding unnecessary formatting work for disabled levels.

- ConsoleSink and FileSink are installed by Game during startup.
- ILogSink lets a project add another destination such as a database, telemetry bridge, or custom in-game viewer.
- The queue is bounded so logging cannot grow memory without limit.
- Sink failures are isolated so one broken sink does not prevent the others from receiving logs.
- Fatal calls flush queued logs. Critical calls write crash-style output, flush, and then throw.
- The file sink handles daily rotation, size rollover, retention, and UTF-8 byte-size tracking.

Beginner tip

Use categories such as "Assets", "Renderer", or "Gameplay" consistently. Good categories turn a large log file into something you can actually search during debugging.

CHAPTER 27

27. Math and Utility Types

VOID owns common 2D math types so high-level engine code is not tied to a renderer library. This became especially important after removing SFML.

Core math

Type	Purpose
Vect2	2D vectors, positions, directions, arithmetic, normalization, interpolation, rotation, snapping, and related helpers.
Vect3 / Vect4	Higher-dimensional vector values used where appropriate.
Rect2	2D rectangles, containment, intersection, union, offset/inflate operations, and geometry helpers.
Color	RGBA color values, named colors, interpolation, alpha helpers, and parsing.
Matrix	VOID-owned 4x4 matrix math used by cameras and renderer transforms.
Easing	41 normalized animation curves.
FastRandom	Fast random helpers with documented inclusive/exclusive range behavior.

Helpers

The repository also includes helpers for hashing, alignment, collision tests, files, JSON, maps, strings, numbers, sounds, text, runtime instance creation, and type discovery. HashHelper provides FNV-1a based 32-bit and 64-bit hashing used across several systems.

CHAPTER 28

28. Discovery, Plugins, and Extensibility

VOID has two complementary ideas: discover components at runtime, and expose stable interfaces where replacement implementations make sense.

Discoverable types

DiscoverableHelper scans loaded assemblies for classes marked with DiscoverableAttribute. It supports filtering by base type/interface, name, category, and priority. Scan modes include All, ExcludeFramework, Custom, Whitelist, and Blacklist. Caches are invalidated when new assemblies are loaded.

Important extension points

Extension point	What a developer can replace or add
IAsset	Custom runtime asset type
IMount	Custom asset source
IAtlasPacker	Texture packing strategy
ILogSink	Logging destination
IRendererBackend	Complete graphics backend
IGraphicsDevice	Backend GPU resource/draw implementation
IBatcher	Custom batching strategy
IRenderTarget	Custom render surface
BaseCamera	Specialized camera behavior
ContentTypeWriterReader	Game-specific save data format

The important design rule

Extension points are most useful when the game-facing API remains stable. The goal is to let a developer swap a low-level implementation without forcing normal game code to understand that implementation.

CHAPTER 29

29. Performance Model and Hot Paths

VOID does not try to hide performance concerns. Several systems are designed around reducing repeated allocation, redundant API calls, and unnecessary state changes.

Examples in the current code

- **Sprite batching:** indexed quads reduce duplicate vertices, and compatible sprites share draw submissions.
- **OpenGL state cache:** avoids rebinding unchanged shaders, textures, VAOs, index buffers, viewports, clear colors, and blend state.
- **Texture atlas:** reduces texture switches, tracks region use per batch, and performs incremental defragmentation.
- **Pathfinding:** reuses arrays and heap/queue storage across queries.
- **Audio:** reuses a fixed pool of SoundInstance objects.
- **Input:** device and action state are represented as stable snapshots for an update.
- **Logging:** asynchronous queueing keeps normal game code from doing file I/O for every log call.

Performance statistics

BatchStats reports CPU-side batch timing and draw statistics. A previous implementation mislabeled CPU stopwatch timing as GPU time; current code reports CPUTime correctly. GPUTime is retained for API compatibility until a real GPU timing implementation is added.

Beginner rule

Optimize the real bottleneck. Batching, pooling, and caching are useful, but measure your game before making complex changes purely because something sounds faster.

CHAPTER 30

30. What VOID Deliberately Does Not Try to Be

A clear boundary is part of the engine design. VOID is meant to provide framework-level building blocks, not every subsystem a game could possibly need.

Intentional scope

The project documentation explicitly leaves full physics and full UI frameworks to the game or to libraries chosen by the developer. That keeps the core smaller and avoids forcing every project into one physics model or one widget system.

The same philosophy applies more broadly: use the built-in systems when they fit, extend them when a variation is reasonable, and bring another library when a specialized problem is better solved outside the engine.

Why this helps novices

A smaller framework surface can be easier to understand. You can learn one system at a time instead of learning a giant editor, scene format, component model, scripting layer, and rendering stack all at once.

CHAPTER 31

31. Packages, Dependencies, and Platform Support

This chapter separates the engine package from the optional tooling packages so it is clear what ships where.

Engine package

Dependency	Version / role
Void.Engine	2.1.0
.NET	net10.0
SDL3-CS	3.4.16 - platform/window/input layer
Silk.NET.OpenGL	2.23.0 - built-in renderer
Silk.NET.OpenAL	2.23.0 - audio API
Silk.NET.OpenAL.Soft.Native	1.23.1 - OpenAL Soft native runtime
NAudio.SoundFile	3.0.1 - sound decoding
StbImageSharp	2.30.16 - image decoding
Void.Packer	Project/package used for asset archives

Tooling packages

Package	Current package version / purpose
Void.Packer	1.0.1 - pack archive library
Void.Packer.CLI	1.0.1 - dotnet global tool, command name void-packer
Void.Templates	1.0.1 - dotnet project template package

Supported desktop platforms

Platform	Status
Windows	Supported
macOS	Supported
Linux	Supported

CHAPTER 32

32. Building and Shipping a Game

Running from your development machine is only the first half of the workflow. A finished game needs a publish build, its content, and the native runtime dependencies required by the selected platform.

Development versus publish

```
dotnet run

# Example framework-dependent release publish
dotnet publish -c Release

# Example platform-specific publish
dotnet publish -c Release -r win-x64
dotnet publish -c Release -r linux-x64
dotnet publish -c Release -r osx-x64
```

The exact publish flags depend on whether you want a framework-dependent build, a self-contained build, trimming, single-file output, and which runtime identifier you target. Test the exact output you plan to distribute rather than assuming a development build proves the published package is complete.

Content deployment

- Keep normal Content or Assets files beside the published game when using filesystem mounts.
- Use Void.Packer when you want archive-based shipped content and mount those packs at runtime.
- Make sure build output preserves the relative paths your asset code expects.
- Do not forget native dependencies used by SDL3, OpenAL Soft, and the selected renderer/runtime platform.

Platform-specific testing

A build that launches on your development platform does not automatically prove another operating system will behave the same. Test fullscreen behavior, display selection, controller mappings, audio initialization, file paths, case sensitivity, and Linux window-backend behavior on the target platform itself.

A release checklist

- Publish a Release build for the intended runtime identifier.
- Launch the published output outside the IDE.
- Verify assets and packs load from a clean directory.
- Verify logs and saves are written where you expect.
- Test keyboard, mouse, and at least the gamepads you officially support.
- Test audio, fullscreen/windowed transitions, resizing, and high-DPI/content-scale behavior.
- Keep symbols or a matching build archive somewhere you control so crash logs can be traced back to the shipped version.

Beginner rule

Do a clean-machine-style test before calling a build finished. Missing native files and missing content are often invisible on a development machine because your source tree happens to contain them.

CHAPTER 33

33. VOID 2.1.0 Release Highlights

Version 2.1.0 focuses on timing correctness, camera behavior, coroutine internals, and Beacon usability while keeping the 2.0 renderer/platform architecture intact.

Camera

- Cameras attached to a batcher update automatically during rendering.
- `BaseCamera.OnUpdate(FrameTime)` supports custom time-based camera logic.
- A shared camera updates at most once per rendered frame.

Frame timing

- Fixed updates use the configured fixed interval.
- Rendering uses actual rendered-frame elapsed time even in fixed-step mode.
- `FrameTime` continues to expose time scale, total time, accumulator, interpolation alpha, FPS, and slow-frame tracking.

Coroutines

- VOID-native `IEnumerator` runner with nested child chaining.
- Root handles remain valid while children run.
- Stopping a root cleans up the active child chain.
- Starting and stopping during an active coroutine update is safe.

Beacons

- `BeaconHandle` payload is read-only and default-safe.
- Subscription mutation is synchronized while publishing remains synchronous and direct.
- `PublishDelay` and typed payload helpers were added.

CHAPTER 34

34. Migration Notes: Older VOID to 2.1.0

Older VOID documentation described a substantially different engine. This page summarizes the conceptual changes that matter when updating old code or old notes.

Older assumption	Current 2.1.0 direction
SFML window / rendering types	SDL3 platform layer plus renderer-neutral graphics contracts.
SFML renderer is the engine architecture	OpenGL is only the built-in IRendererBackend implementation.
SFML audio path	OpenAL / OpenAL Soft playback with NAudio.SoundFile decoding.
Renderer-specific math conversions	VOID-owned Vect, Rect, Color, and Matrix types at engine level.
Camera updates performed manually by game code	Attached cameras update automatically during rendering in 2.1.0.
Input queries advance action state	InputAction.GetState returns the already-created update snapshot.
Old coroutine internals	VOID-native IEnumerator chaining with safe nested cleanup and root handles.

Treat current source and current package documentation as authoritative. When an old example conflicts with the renderer-neutral or snapshot-based architecture described here, update the example rather than preserving the old SFML-era behavior.

Best migration strategy

Move one boundary at a time: startup/windowing, renderer-facing types, assets, input, audio, then gameplay systems. Keeping gameplay rules separate from platform and renderer code makes this much easier.

CHAPTER 35

35. Beginner Workflow: How the Systems Fit Together

This is a practical way to think about a small VOID game without needing to understand every subsystem on day one.

A simple project flow

Step	What you do
1. Configure	Build GameSettings for window size, viewport, timing, renderer, audio limits, atlas settings, and app folders.
2. Load content	Use AssetManager for textures, sounds, fonts, spritesheets, LDtk maps, and shaders.
3. Bind controls	Define InputAction entries that map game concepts such as Jump or Pause to devices.
4. Update	Read the current input snapshot and update game logic in OnUpdate.
5. Draw	Use SpriteBatcher and PrimitiveBatcher in OnDraw. Add a camera when world-space movement needs a view transform.
6. Add flow	Use coroutines for timed sequences, StateMachine for named modes, and Beacons for decoupled events.
7. Persist	Use ContentTypeWriterReader for saves and Void.Packer for shipped assets.
8. Diagnose	Use Logger categories and atlas/batch metrics when debugging behavior or performance.

Examples in the repository

Example	What to use it for
Examples/FlappyBirb	A small project to trace startup, a Game subclass, input, assets, drawing, and a complete simple game loop without much unrelated code.
Examples/Scavengers	A larger example to study how more systems are organized once a project grows beyond a tiny sample.

When learning an API, start from the smallest example that demonstrates the behavior you need. Reading a complete small game is often easier than searching a large project for one call in isolation.

You do not need everything at once

A first VOID project can be just Game, GameSettings, InputAction, AssetManager, and SpriteBatcher. The other systems are there when the game grows into problems that need them.

CHAPTER 36

36. Common VOID Recipes

These short recipes connect the individual systems into the small tasks a new project performs repeatedly. They are starting patterns, not rules that every game must copy exactly.

Create settings and run the game

```
var settings = GameSettings.Instance
    .SetName("MyGame")
    .SetAppCompany("MyStudio")
    .SetWindow(1280, 720)
    .SetViewport(320, 180)
    .Build();

using var game = new MyGame(settings);
game.Run();
```

Load a texture

```
Texture player = AssetManager.Instance
    .Load<Texture>("Sprites/player.png");
```

Create and read a named input action

```
InputAction.AddAction("Jump")
    .AddKey(KeyboardKey.Space)
    .AddGamepad(GamepadButton.A);

var input = InputAction.GetState();
if (input.IsJustPressed("Jump"))
    StartJump();
```

Draw sprites and text

```
sprites.Begin(camera: camera);
sprites.Draw(player, position, Color.White);
sprites.DrawText(font, "Score: 10", new Vect2(8, 8), Color.White);
sprites.End();
```

Convert pointer coordinates into the world

```
Vect2 worldPosition = camera.ScreenToWorld(screenPosition);
```

Use a coroutine for a timed sequence

```
IEnumerator FlashMessage()
{
    _showMessage = true;
    yield return 1.5f;
    _showMessage = false;
}
```

Try a save operation without exceptions for normal failure

Prefer the save system's TrySave and TryLoad APIs when a missing file, bad version, wrong key, or other expected failure should be handled as a normal branch. Inspect the returned SaveError so the game can display the correct message instead of treating every failure as a crash.

Send a decoupled event

Use BeaconManager when one system should announce that something happened without knowing all the listeners. Keep handlers short because Publish is synchronous on the calling thread.

Recipe rule

Copy the shape of a recipe, not necessarily every name or lifetime choice. Your real owner object, asset path, action names, and update structure should match your game.

CHAPTER 37

37. Troubleshooting and FAQ

When something does not work, narrow the problem to one system first. VOID's logs, settings, metrics, and explicit subsystem boundaries are meant to make that possible.

Window opens but nothing draws

- Confirm OnDraw is actually running and the game has not already requested shutdown.
- Confirm Begin and End are paired for the batcher you are using.
- Check sprite position, tint alpha, source region, depth/sort expectations, and whether the camera can see the object.
- Temporarily draw a primitive or built-in text at a known viewport position to separate asset problems from rendering problems.

A texture or other asset does not load

- Check the content root, mount order, exact relative path, and filename case. Linux filesystems commonly expose case mistakes that Windows can hide.
- Check the log for the asset category and verify the file exists in the published output or mounted pack.
- If using a pack, verify the archive with void-packer and confirm the pack mount was added before the load request.

Input does not respond

- Make sure the action was registered before reading it.
- Remember that GetState returns a snapshot; it does not update devices on demand.
- Check focus behavior if IgnoreInputWhenUnfocused is enabled.
- For controllers, confirm the pad is connected, mapped, and bound to the action you are testing.

Sound does not play

- Confirm the Sound asset loaded and the OpenAL audio system initialized successfully.
- Check master/category/instance volume, looping state, and whether the sound was immediately stopped or recycled.
- If many sounds are active, remember the pool has a finite audio limit and may steal a lower-priority or long-playing voice.

The image is stretched, cropped, or blurry

Check the relationship between viewport size, window size, and WindowScaleMode. Pixel art usually wants an internal resolution chosen deliberately plus PixelPerfect or another scaling strategy that preserves the intended aspect ratio.

A custom shader or render target looks wrong

Reduce the pass to the simplest possible case. Verify target size, clear behavior, shader inputs, texture binding, and final presentation separately. Draw the unprocessed source target directly before debugging the post-processing step.

Fullscreen or Linux window behavior differs

Check whether the game is using desktop or exclusive fullscreen and which Linux backend is active. X11, XWayland, and native Wayland do not expose every native-window behavior in exactly the same way.

Where are logs, saves, and config files?

GameSettings controls whether application data uses the platform application-data location or a local game directory. The default subfolder names are Logs, Saves, Config, and Temp. Game exposes the resolved application folders after startup.

FAQ

Question	Short answer
Do I need to understand OpenGL to make a normal VOID game?	No. Normal game code uses renderer-neutral engine APIs and high-level batchers.
Do I have to use the texture atlas?	No. Sprite rendering can participate in the atlas or bypass it where appropriate.
Do I need a camera for every game?	No. One-screen games can draw directly in viewport space.
Are coroutines background threads?	No. They are frame-driven routines resumed by the game update flow.
Should missing input actions throw?	No. TryGetAction is the explicit safe lookup, and missing/default action state behaves as released.
Does VOID include a full physics engine or full UI toolkit?	No. Those are intentionally outside the core scope.

Debug from the boundary inward

First decide whether the failure is configuration, asset lookup, input, gameplay logic, rendering, audio, or platform behavior. Then inspect only that boundary and its logs instead of changing several systems at once.

CHAPTER 38

38. Source Map: Where to Look in the Repository

When the manual is not enough, these folders and files are the best starting points for reading the implementation.

Area	Primary source location
Game lifecycle	Src/Void.Engine/Game.cs
Configuration	Src/Void.Engine/GameSettings.cs
Renderer contracts	Src/Void.Engine/Graphics/Rendering/
Built-in OpenGL	Src/Void.Engine/Graphics/Rendering/OpenGL/
Batching	Src/Void.Engine/Graphics/BaseBatcher.cs, SpriteBatcher.cs, PrimitiveBatcher.cs
Atlas	Src/Void.Engine/Graphics/Atlas/
Cameras	Src/Void.Engine/Cameras/
Assets	Src/Void.Engine/Assets/AssetManager.cs and Assets/Mounts/
Input	Src/Void.Engine/Inputs/
Audio	Src/Void.Engine/Audio/ and Sounds/
Saves	Src/Void.Engine/Saves/
Coroutines	Src/Void.Engine/Coroutines/
Beacons	Src/Void.Engine/Beacons/
State machine	Src/Void.Engine/FSM/StateMachine.cs
Pathfinding	Src/Void.Engine/Pathfinding/
Logging	Src/Void.Engine/Logs/
LDtk	Src/Void.Engine/Assets/Loaders/LDtk/
Discovery	Src/Void.Engine/Helpers/DiscoverableHelper.cs
Packer	Src/Void.Packer/ and Src/Void.Packer.CLI/
Templates	Src/Void.Templates/

CHAPTER 39

39. Glossary

A short dictionary for engine terms used throughout this document.

Asset

A runtime resource such as a texture, sound, map, font, spritesheet, or shader.

Mount

A source that AssetManager searches for files, such as a normal folder or a pack archive.

Backend

The implementation that talks to a specific low-level API, such as the built-in OpenGL renderer.

Batch / Batchers

A way to collect many draw requests and submit compatible work together to reduce graphics overhead.

Buffer

GPU memory used for structured data such as vertices or indices.

Camera

A transform that controls how world-space coordinates are viewed on the screen.

Coroutine

A frame-driven routine that can pause and resume across updates without blocking the game thread.

Dead zone

A small analog input range ignored to prevent stick or trigger noise from counting as intentional input.

Flow field

Pathfinding information that many agents can follow toward the same destination.

Frame

One displayed image produced by the game loop.

Heuristic

An estimate used by pathfinding to decide which direction is promising.

LRU

Least Recently Used, a policy that evicts old cached entries before recently used ones.

Render target

An off-screen surface that receives drawing and can later be used as a texture.

Renderer-neutral

An API that describes what the engine needs without exposing a specific graphics API to normal game code.

Shader

A small GPU program used during rendering to transform vertices or calculate pixel colors.

Snapshot

A read-only view of state captured at a particular update, commonly used for input.

Texture atlas

A larger texture page containing many smaller sprite regions.

Tween

A time-based interpolation from one value to another, usually shaped by an easing curve.

VSync

Vertical synchronization, which coordinates presentation with the display refresh cycle to reduce visible tearing.

VOID Engine: the foundation is yours. Build the rest your way.

Document target: VOID Engine 2.1.0, source snapshot ee4aab287a8c5459f123fbacfa59c873a0bb77d7.